Ask the LLM to review your code

```python
client = genai.Client(api_key=api_key)
model = "gemini-3-flash-preview"
prompt = "Generate pairs of numbers (x and y coordinates) for points on a circle. " \
        "Print only coordinates nothing more. I need output: [(1,2), (2,4)]"

response = client.models.generate_content(
    model=model,
    contents=prompt,
)

import ast

points = ast.literal_eval(response.text)

import matplotlib.pyplot as plt
```

✅ **Fix: Enforce Structured Output**

Use JSON and validate:

Python

```python
import json

prompt = """
Generate 20 coordinate pairs (x, y) for points on a circle.
Return ONLY valid JSON in this format:
[[x1, y1], [x2, y2], ...]
No explanations.
"""
```

Then:

Python

```python
points = json.loads(response.text)
```

This is much more reliable.

https://chatgpt.com/s/t_69a0068e72e881918cf180b80e45dfeb

Ask the LLM to explain your code.

⚠️ Potential Issues

1️⃣ Model Output Might Break Parsing

If Gemini outputs:

‹/› Code

```
Here are your points:
[(1,0), (0,1)]
```

`ast.literal_eval()` will fail.

Better approach:

- Use stricter prompting
- Or parse more defensively

2️⃣ You Are Trusting the AI for Math

The model may generate:

- Points not perfectly on a circle
- Irregular spacing
- Too few points

It's not mathematically guaranteed.

https://chatgpt.com/s/t_69a0068e72e881918cf180b80e45dfeb

Ask the LLM to add comments to your code.

Here is your code with clear, descriptive comments added:

Python

```python
# Import the Google Generative AI client library
from google import genai
from google.genai import types

# Read your API key from a local file (make sure this file exists and is se
api_key = open("LLM/gemini.txt").read().strip()

# Create a client object using your API key
client = genai.Client(api_key=api_key)

# Specify which Gemini model to use
model = "gemini-3-flash-preview"

# Define the prompt that will be sent to the model
# We ask the model to generate coordinate pairs for points on a circle
# and return ONLY a Python-style list of tuples
prompt = "Generate pairs of numbers (x and y coordinates) for points on a c

# Send the prompt to the model and generate a response
response = client.models.generate_content(
    model=model,
    contents=prompt,
)

# Import ast to safely evaluate the string output as Python data
import ast

# Convert the model's text response into a Python list of tuples
# literal_eval safely parses strings like "[(1,2), (2,4)]"
points = ast.literal_eval(response.text)

# Import matplotlib for plotting the points
import matplotlib.pyplot as plt

# Extract x and y coordinates into separate lists
x = [p[0] for p in points]
y = [p[1] for p in points]

# Append the first point again at the end to close the circle visually
x.append(points[0][0])
y.append(points[0][1])

# Create a new figure for plotting
plt.figure()

# Plot the points and connect them with lines
plt.plot(x, y)

# Display the plot window
plt.show()
```

https://chatgpt.com/s/t_69a0081f0cf88191a352298ad4f362cd

Ask the LLM to check if your code complies with the PEP8 coding style and highlight where it does not.

Current code:

Python

```python
from google import genai
from google.genai import types
...
import ast
...
import matplotlib.pyplot as plt
```

Problems:

- Imports should be at the top of the file.
- Standard library imports should come first.
- Third-party imports should follow.
- One blank line between import groups.

Correct Order:

Python

```python
import ast

import matplotlib.pyplot as plt
from google import genai
from google.genai import types
```

https://chatgpt.com/s/t_69a0086f399881919c2ed25ec31ec3a5

Ask the LLM to apply clean code principles to your code.

```python
api_key = open("LLM/gemini.txt").read().strip()

client = genai.Client(api_key=api_key)
model = "gemini-3-flash-preview"
prompt = "Generate pairs of numbers (x and y coordinates) for points on a circle. " \
        "Print only coordinates nothing more. I need output: [(1,2), (2,4)]"

response = client.models.generate_content(
    model=model,
    contents=prompt,
)

import ast
```

```python
# ========================
# Configuration
# ========================
API_KEY_PATH = "LLM/gemini.txt"
MODEL_NAME = "gemini-3-flash-preview"
PROMPT = (
    "Generate pairs of numbers (x and y coordinates) for points on a circle. "
    "Print only coordinates nothing more. "
    "I need output: [(1,2), (2,4)]"
)


# ========================
# Core Logic
# ========================
def load_api_key(path: str) -> str:
    """Load API key from file."""
    try:
        with open(path, "r") as file:
            return file.read().strip()
    except FileNotFoundError as error:
        raise FileNotFoundError(f"API key file not found at: {path}") from error
```

https://chatgpt.com/s/t_69a008e937148191b44734c5e8c7864c

Ask the LLM to optimize your code and provide suggestions on how to make it more efficient or faster.

```python
response = client.models.generate_content(
    model=model,
    contents=prompt,
)

import ast

points = ast.literal_eval(response.text)

import matplotlib.pyplot as plt

x = [p[0] for p in points]
y = [p[1] for p in points]

x.append(points[0][0])
y.append(points[0][1])
```

⚡ Performance Improvements

1️⃣ Avoid LLM for deterministic math

If your goal is simply to generate circle points, using an LLM is **massively slower** and unnecessary.

A pure Python + NumPy solution is 1000x faster and deterministic:

‹/› Python

```python
import numpy as np
import matplotlib.pyplot as plt

theta = np.linspace(0, 2*np.pi, 100)
x = np.cos(theta)
y = np.sin(theta)

plt.plot(x, y)
plt.axis("equal")
plt.show()
```

👉 This removes:

- API latency
- Parsing overhead
- Network dependency
- Cost

https://chatgpt.com/s/t_69a009c0cfe48191a8435a365acca1f0